

这种撤销、持久化、重做的方式不仅能够打破环形依赖、保证持久化顺利进行，还能解决写回迟滞的问题。若某个结构被频繁修改，导致不断有新的依赖产生，Soft updates 可以将部分新的修改撤销，在快速完成持久化后将修改重做，以此避免新依赖不断推迟该结构上修改的持久化。

12.5.4 文件系统恢复

在发生崩溃后，文件系统一般需要通过 12.2 节介绍的文件系统检查 (fsck) 来保证文件系统的一致性。在 fsck 完成之前，文件系统一般处于无法使用状态，以保证潜在的非一致性问题不会造成更加严重的数据损坏。然而 fsck 所需要的时间与文件系统的大小相关。这将导致在崩溃之后的很长一段时间内，文件系统都处于不可用状态。对于 Soft updates 来说，发生崩溃之后同样有可能产生不一致。但是由于 Soft updates 在写回时严格按照依赖顺序，在崩溃后，Soft updates 文件中存在的不一致并不会导致文件系统无法使用或导致其中的数据发生损坏。一些可能发生的不一致情况包括：

- 未使用的块没有出现在空闲空间列表中。
- 未被引用的 inode 没有出现在空闲 inode 列表中。
- inode 中保存的链接数可能超过实际链接数（如 inode 中链接数为 3，却只有 2 个目录项指向此 inode），从而导致该 inode 及其引用的数据块无法被回收。

即使发生了这些不一致情况，Soft updates 依然可以在挂载后立即开始处理新的文件系统请求，无须等待耗时的文件系统检查。管理员可以选择在合适的时刻执行 Soft updates 的文件系统检查操作来修复这些问题。

12.5.5 讨论和小结

总的来说，Soft updates 通过记录和保持文件系统结构之间的持久化依赖关系，保证各个结构按照既定的顺序进行持久化，从而在异步写回的情况下避免在存储设备上产生严重的不一致情况。异步写回使得 Soft updates 能够以内存的性能完成文件系统操作；而严格的写回（持久化）顺序令 Soft updates 在崩溃之后可以立即投入使用，无须等待长时间的恢复或者检查。对于可能出现的一些“良性”不一致情况，用户可以在适当的时刻通过文件系统检查进行修复，以此来保证整个文件系统的一致性。

Soft updates 是不同于日志、写时拷贝等技术的保证文件系统一致性的方法，其设计上避免了对文件系统存储格式的修改，使得文件系统的设计与一致性保证解耦。同时，其避免了日志的两次写入和写时拷贝的写放大等性能问题。然而，Soft updates 也并非完美的。为了实现 Soft updates 中涉及的各种机制，文件系统需要在内存中记录大量的依赖关系。这会用到大量的辅助结构，导致 Soft updates 的实现非常复杂。因此，在实际使用中，Soft updates 还经常与日志等机制结合起来使用^[14]，以简化系统的设计与实现。

12.6 案例分析：日志结构文件系统

本节主要知识点

- 日志结构文件系统是如何保存文件的数据和元数据的？
- 日志结构文件系统如何管理空间？
- 日志结构文件系统与使用日志的文件系统有何区别？有何优势？
- 日志结构文件系统如何保证文件系统崩溃一致性？

本章在介绍重做日志时，曾提到重做日志不急于将日志内容写回原位置。那么是否能够更极端一些，直接将整个文件系统以一种日志的形式保存呢？这样每个文件系统结构都没有“原位置”，也就不需要对日志内容进行写回操作了。日志结构文件系统（Log-structured File System, LFS）正是这种主要以日志结构保存数据和元数据的文件系统。在本节中，我们将基于最早的 Sprite LFS^[15]对日志结构文件系统的基本概念和原理进行介绍。

小知识：最初 LFS 设计的假设

最初的 LFS 设计基于这样一个假设：由于文件系统数据可以被缓存在内存中，大多数文件系统读请求可以在内存中完成。随着内存容量的提升，文件系统的读取操作可以达到比较好的性能。在这种情况下，对于存储设备的访问将主要被写请求占据。考虑到机械硬盘顺序写入可以避免磁盘寻道时间，以日志的形式组织文件系统可以让写入性能达到最优。另外需要注意的是日志与日志结构这两种技术是有区别的。日志一般是临时性的，日志中记录的内容最终会被写入它们原本应该在的位置；而在使用日志结构时，数据并没有固定的保存位置。

12.6.1 基本概念与空间布局

在此前介绍的文件系统中，每个结构在存储设备上均有固定的存储位置。当某个结构中的数据被修改时，其修改被“原地”写入，该结构的位置并不会发生变化。当多个结构被修改时，若这些结构的固定位置不能被一个原子修改操作覆盖，文件系统需要向存储设备发出多次随机写入请求。由于存储设备的随机写入性能一般要比顺序写入性能差，大量的随机写入请求会降低文件系统的总体性能。

为了减少随机写入，LFS 以日志的形式保存整个文件系统的数据。所有的修改均会追加到日志空间的尾部，从而保证对于存储设备的顺序写入。由于使用这种写入方式，日志中保存的文件结构和数据并没有固定的“原位置”。随着文件系统的使用，每个结构的保存位置也会不断发生变化。那么如何才能在不断变化位置的结构中找到某个特定的

结构呢？

事实上，在 LFS 中，一些重要结构的位置是固定的。表 12.3 中给出了 LFS 中所使用的各种结构。其中，超级块和检查点（Checkpoint）是两个固定位置的结构。超级块是文件系统存储格式中非常重要的结构。文件系统在挂载时，需要读取文件系统的超级块以区分不同的文件系统，并获知一些文件系统的全局元数据。为了让操作系统在挂载时更加方便地识别文件系统类型，超级块的位置被设计成固定的。另一个固定位置的结构为检查点，其中保存了文件系统其他重要结构的位置，例如 inode 表所在的位置等。通过从固定位置的检查点中找到这些结构的位置，我们可以“顺藤摸瓜”找到文件系统的所有其他结构和数据。

表 12.3 LFS 中的各种结构

结构名称	位置	说 明
超级块	固定	保存文件系统的全局元数据等信息
检查点	固定	保存检查点
inode 表	日志	表示 inode 的位置、分配和使用情况
inode	日志	inode 结构
间接块	日志	即索引块，用于保存其他间接块或者索引块的位置
数据块	日志	保存文件数据、目录的目录项等
段概要	日志	记录段中的有效数据
段使用表	日志	记录段中的有效字节数、段的最后修改时间等
目录修改日志	日志	保存对目录进行的操作，保证文件链接数的一致性

图 12.11 展示了一个简单的 LFS 存储布局。在此文件系统中，超级块保存在存储空间的开头。之后为检查点区域，包括两个检查点。再之后为日志区域，其以段（Segment）为单位进行管理，保存文件系统中除超级块和检查点之外的所有结构和数据。关于基于段的空间管理和 LFS 中新增的检查点区域、段概要、段使用表等结构，我们将在后面的小节中进行介绍。



图 12.11 LFS 的存储布局 [15]

12.6.2 数据访问与操作

图 12.12 展示了在 LFS 中创建文件的过程。块 15 到块 18（即两条虚线之间）为创建文件“g”时新增的日志内容。我们先来看一下创建文件“g”之前 LFS 中的状态（即第一条虚线左边的日志内容）。块 14 中保存了一个 inode 表结构。inode 表中记录了每个 inode 号是否有效，以及每个有效 inode 号所对应的 inode 结构的位置。块 14 的 inode 表

中记录了一个 inode 号为 2 的 inode 及其位置，通过读取 inode 表中的内容，可以找到该 inode 结构在块 13 中[Ⓐ]。通过读取块 13 中 inode 结构中的元数据，可以得知该 inode 为目录 inode，其中包括三个数据块用于保存目录项——块 10、块 11 和块 12。如果依次读取每个数据块中保存的目录项，可以发现该目录中保存了四个有效目录项：文件“a”对应 6 号 inode，文件“b”对应 9 号 inode，文件“e”对应 5 号 inode，文件“f”对应 3 号 inode。通过这些有效目录项中记录的 inode 号，文件系统就可以在块 14 中的 inode 表找到对应的 inode 结构，从而访问这些文件的内容。2 号 inode 保存的目录项中还包括一个无效目录项，从中可以看出文件名为“c”，这说明该目录中很可能曾经保存过文件“c”，但是该文件目前已经不在这个目录中了（被删除或者被移出该目录）。

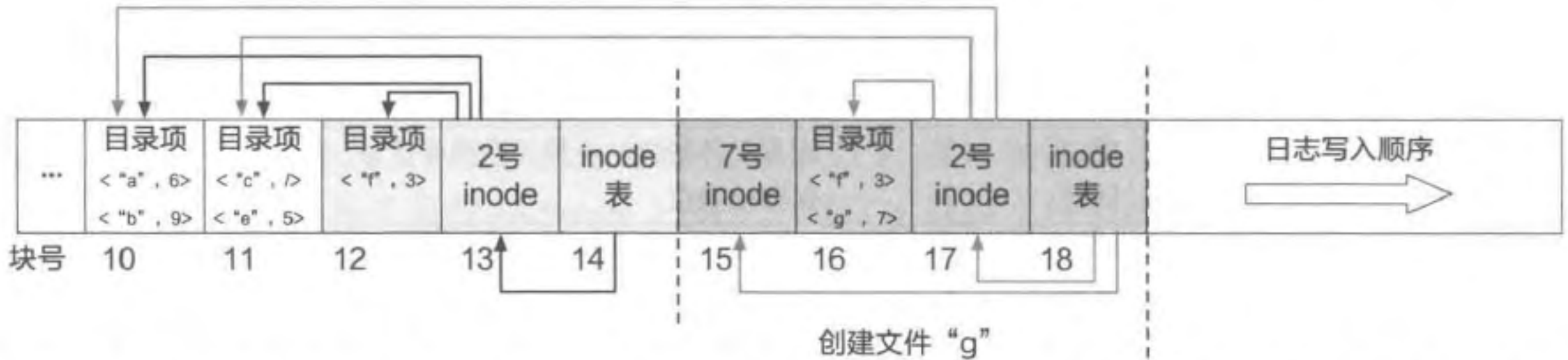


图 12.12 LFS 中的创建文件过程。新文件的文件名为“g”，inode 号为 7。块 10 和块 11 为在此操作中未做修改的日志。块 12 到块 14 为创建操作中被无效化的日志。块 15 到块 18 为创建操作过程中新增的日志

文件创建

本节依然以此前经常使用的文件创建为例，介绍在 LFS 中如何在存储设备上进行文件操作。在进行文件创建时，至少包括三个比较重要的步骤：分配 inode、初始化 inode 和将新目录项写入父目录中。在图 12.12 中，两条虚线之间新增的日志内容表示这三个步骤对应的修改。

在创建文件“g”时，LFS 首先在日志中追加一个初始化之后的 inode 结构（块 15）。在该 inode 结构的信息被写入日志之后，其处于“游离”状态，换句话说，此时从文件系统根出发扫描整个文件系统，并不会扫描到该 inode 结构。也正因为如此，无论该 inode 是否被写入成功，抑或该 inode 结构只被成功写入了一半，都不会影响文件系统的一致性。

小知识

一般来说，inode 结构的大小要小于块设备的原子写入粒度，因此“inode 结构只被成功写入一半”的情况不会发生。但是对于一些非块存储设备（如非易失性内

Ⓐ 虽然图中将块 13 标记为“2 号 inode”，但实际上 inode 结构中并不记录 inode 号码。

存)的场景来说,这种情况是有可能发生的。另外一个可能出现的场景是一个 inode 结构跨越了两个相邻的块,但是文件系统在设计上一般会避免这种情况的发生。不过,不管对于哪种场景,LFS 在此处崩溃均不会造成文件系统一致性的破坏。

此后,LFS 在日志中追加写入一个数据块(块 16),其内容以 2 号 inode (父目录)中的第三个数据块(块 12)为基础,加入新的目录项,记录文件名“g”对应的 7 号 inode。与此前的 inode 结构相似,该数据块被记录到日志之后同样处于“游离”状态。因此,此数据块的写入同样不影响文件系统的一致性。若此时访问 2 号 inode 结构,则访问到的是操作前的 2 号 inode 结构(块 13),其依然指向三个旧的目录项数据块。

至此,所有的数据修改均已经完成。但是这些修改还都处于“游离”状态,因此接下来的步骤要将这些修改“接入”文件系统树中,以便在发生崩溃之后依然能够看到这些修改后的结构和数据。

根据创建文件时的三个步骤,如果能让 inode 的分配和初始化“生效”,需要更新 inode 表的内容,将新分配的 inode 号与其对应的 inode 结构位置写入表中。此后还需要标记修改后的 inode 表为有效,否则按照修改前的 inode 表是无法访问到修改之后的 inode 结构的。而对于目录项的更新来说,需要修改 2 号 inode 中保存的直接数据块的地址。由于 LFS 是通过追加日志来记录修改的,因此修改 2 号 inode 需要在日志中追加新的 inode 结构,而增加新的 inode 结构之后同样需要修改 inode 表,才能让新的 2 号 inode 结构生效。

为了避免多次写入 inode 表,LFS 先在日志中追加新的 2 号 inode 结构(块 17),其中记录的三个数据块指针包括此前 2 号 inode 结构中已有的两个未修改的数据块(块 10 和块 11),以及刚刚添加了新目录项的数据块(块 16)。新的 2 号 inode 结构追加到日志之后,同样处于“游离”状态,因此不会对一致性产生破坏。

此后 LFS 追加写入日志的结构为 inode 表(块 18),此 inode 表中除了包括原 inode 表中的数据外,还对新加入的 7 号 inode 信息和新修改的 2 号 inode 结构的地址一同进行了记录。一旦新的 inode 表写入完毕,整个创建文件的操作就已经被持久化到 LFS 中。此时如果发生崩溃,LFS 在进行恢复时已经可以将整个创建操作恢复出来。在 LFS 中,通常还会使用检查点等方式简化恢复流程,我们会在后面的小节中进行介绍。

12.6.3 基于段的空间管理

前面的设计都假设整个磁盘空间是一个连续而无限大的日志空间,然而真实存储设备的容量是有限的,当日志空间写入磁盘末尾时,就无法继续顺序写下去。因此,我们不得不考虑重新利用日志空间中被使用过却已经不再有效的空间。

随着对文件系统的不断使用,日志空间部分区域中的数据会由于后来的更新而无效

化，如图 12.12 中的块 12 到块 14。然而直接重用这些空间并不容易，原因有以下两点。第一，日志中的有效数据和无效数据交织混杂在一起。想要重新利用日志中的空间，必须先识别出日志中哪些数据是有效的，哪些数据是无效的，只有无效数据所占用的空间才能被重新利用。第二，日志中无效数据所占据的空间大多是分散而不连续的。如果直接使用这些空间，可能会引入大量随机磁盘写入而造成性能下降，这并不符合 LFS 希望顺序写入日志的初衷。

对于第一个问题，即有效数据识别问题，LFS 可通过增加新的结构记录有效数据块位置的方式来解决。现在我们先假设 LFS 已经能够快速识别日志空间中的有效数据，并以此为前提来看第二个问题。

对于第二个问题，即无效数据占据的空闲空间的组织问题，有两种比较简单的解决方法：

- 空闲链表。这是一种比较直观的方法，其将空闲的空间使用链表连接起来（如图 12.13a 所示）。当文件系统需要使用新的空间时，可以从链表中拿出一块空闲空间，作为接下来的日志空间进行使用；当一段空闲空间使用完之后，再从链表中找出另一块空间继续使用。
- 空间整理。假设数据保存在存储设备 A 上，我们可以再找一个同等容量的设备 B，然后顺序扫描存储设备 A 上的日志空间，将有效数据依次移动到设备 B 的日志空间中。图 12.13b 中展示了这种方法。整理完成后，有效数据全部保存在设备 B 的日志空间的前一半中，无效数据所占用的空间被“移动”到设备 B 的尾部。此后便可以从有效数据的尾部开始，继续以 LFS 的方法进行顺序写入。当然，这种空间整理方法在优化后也可以不借助设备 B 完成。

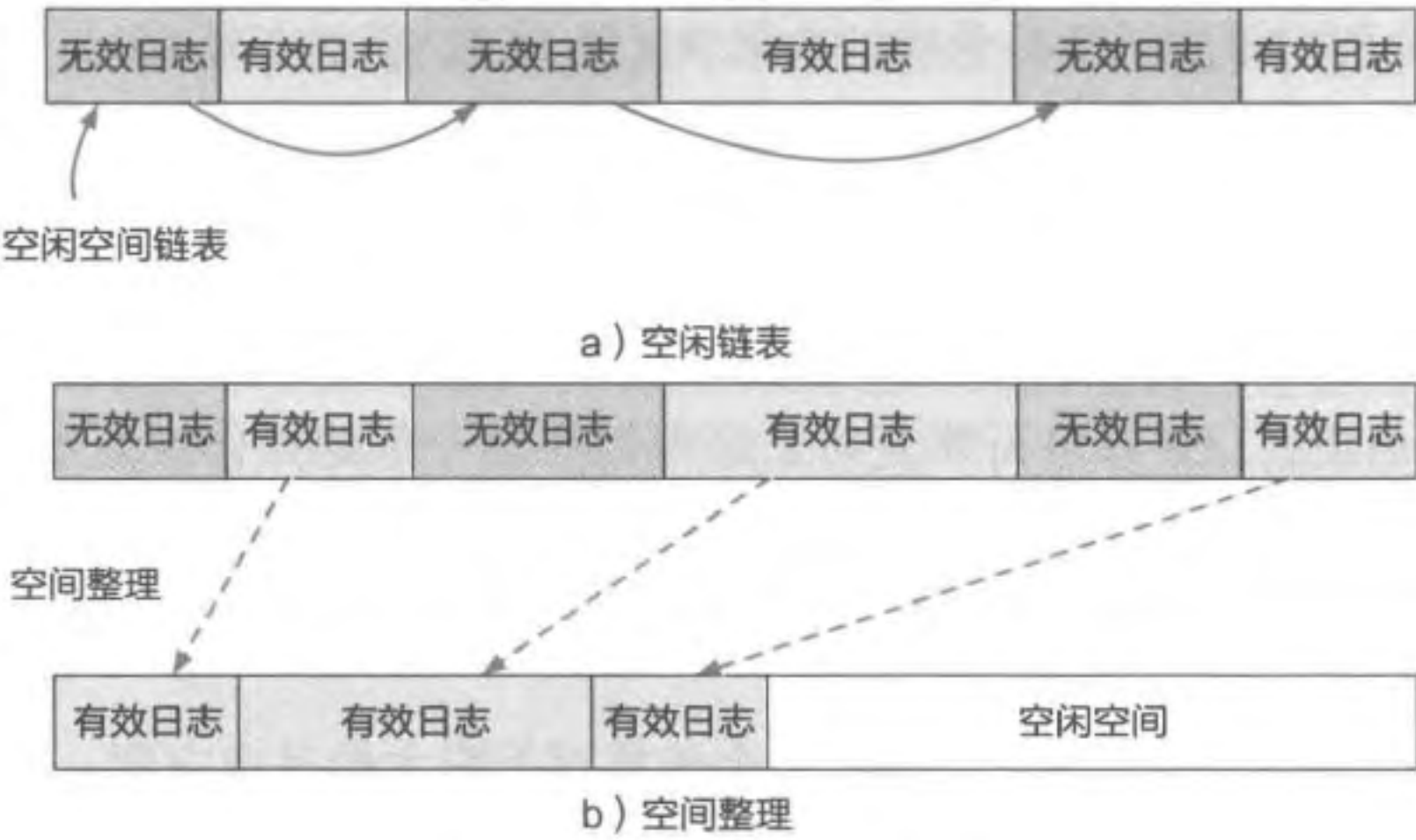


图 12.13 两种空间管理策略

这两种方法各有优劣。使用空闲链表的方法虽然操作比较简单，但是随着文件系统

不断被使用，整个空间越来越“碎片化”，磁盘中大段连续的空闲空间越来越少，顺序写入越来越困难，取而代之的是大量的随机写入，这导致 LFS 的优势完全消失。此外，虽然空闲链表维护起来非常方便，但是用于存放有效数据的有效空间越来越零碎，这导致有效数据段的维护变得越来越复杂且耗时。

相对地，使用空间整理的方法可以保证每次整理后有效数据和空闲区域都是连续的。这保证了 LFS 总是能够进行大量的顺序写操作，从而保持了 LFS 的优势。然而，每次在整理空间时，都需要扫描整个空间，而且由于此过程中需要整理和移动有效数据，整个过程会非常耗时。

为了平衡这两种方法，LFS 提出了段（Segment）的概念，以求既能同时拥有这两种方法的优点，又尽可能避免两者的缺点。图 12.14 给出了段的示意图。在 LFS 中，存储空间被切分成大量固定大小的区域，每个区域被称为一个“段”。每个段是一个连续的日志空间，可以从头到尾顺序记录日志内容。每个段的大小可以进行配置，但是要保证足够大，才能充分发挥存储设备顺序写入的优势。

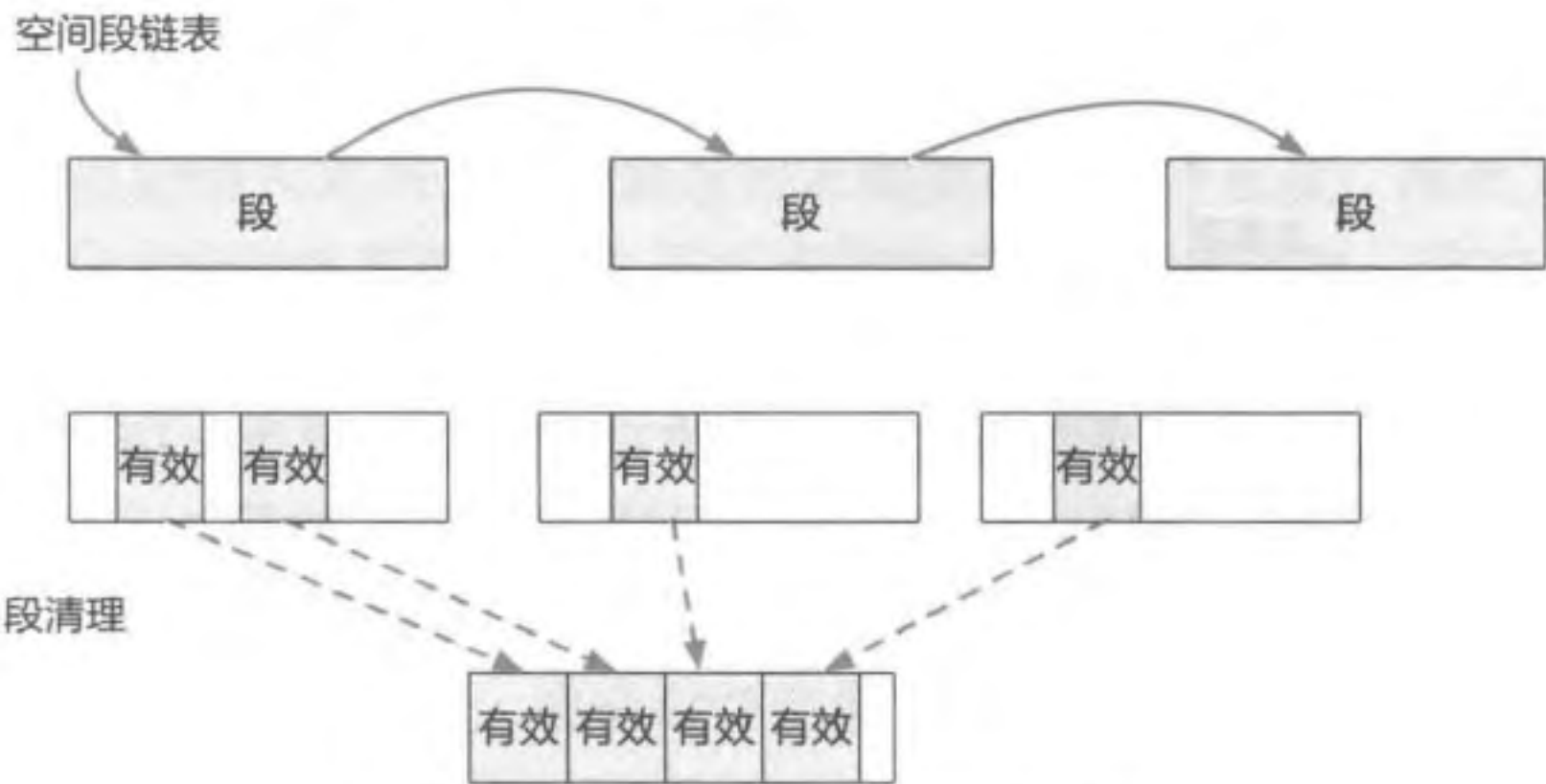


图 12.14 使用段对空间进行管理

LFS 维护了空闲段（即完全不包含任何有效数据的段）的信息，空闲段之间使用空闲链表的方式进行连接（图 12.14 上方的链表）。为了维护每个段内部的空间，段会定期进行清理（图 12.14 的下半部分，对应前文中的空间整理），即将一个或者几个段内的有效数据整理后写入空闲段中。段的清理主要包括以下步骤：

1. 选定一些要进行清理的段，将其内容读入内存。
2. 识别出段中的有效数据。
3. 将有效数据整理后写入空闲段中。
4. 标记被清理的段为空闲，加入空闲链表中。

在了解了段之后，我们回到第一个问题：在使用段的基础上，如何识别每个段中的有效数据呢？

为了进行有效数据识别，LFS 在每个段中增加了一个结构：段概要（Segment

Summary)。如图 12.15 所示，段概要保存在每个段的开头，其中记录了段中的每个块被哪个文件的哪个位置所使用。举例来说，对于一个文件数据块，段概要中会保存其是几号 inode 的第几个数据块，以此来表示其在文件系统树中的位置。在得知段中每个数据块的位置和作用之后，对于段中的任何一个块，LFS 通过判断最新文件系统树中对应位置上的数据块是否为该块，来得知该块中的数据是否依然有效。例如，对于图 12.15 中的块 N，如果通过查找内存中的文件系统结构，得知当前 8 号 inode 的第 2 个数据块并不指向此位置，则可以判断出该块中的数据已经失效，该块为无效块，可以被回收使用。

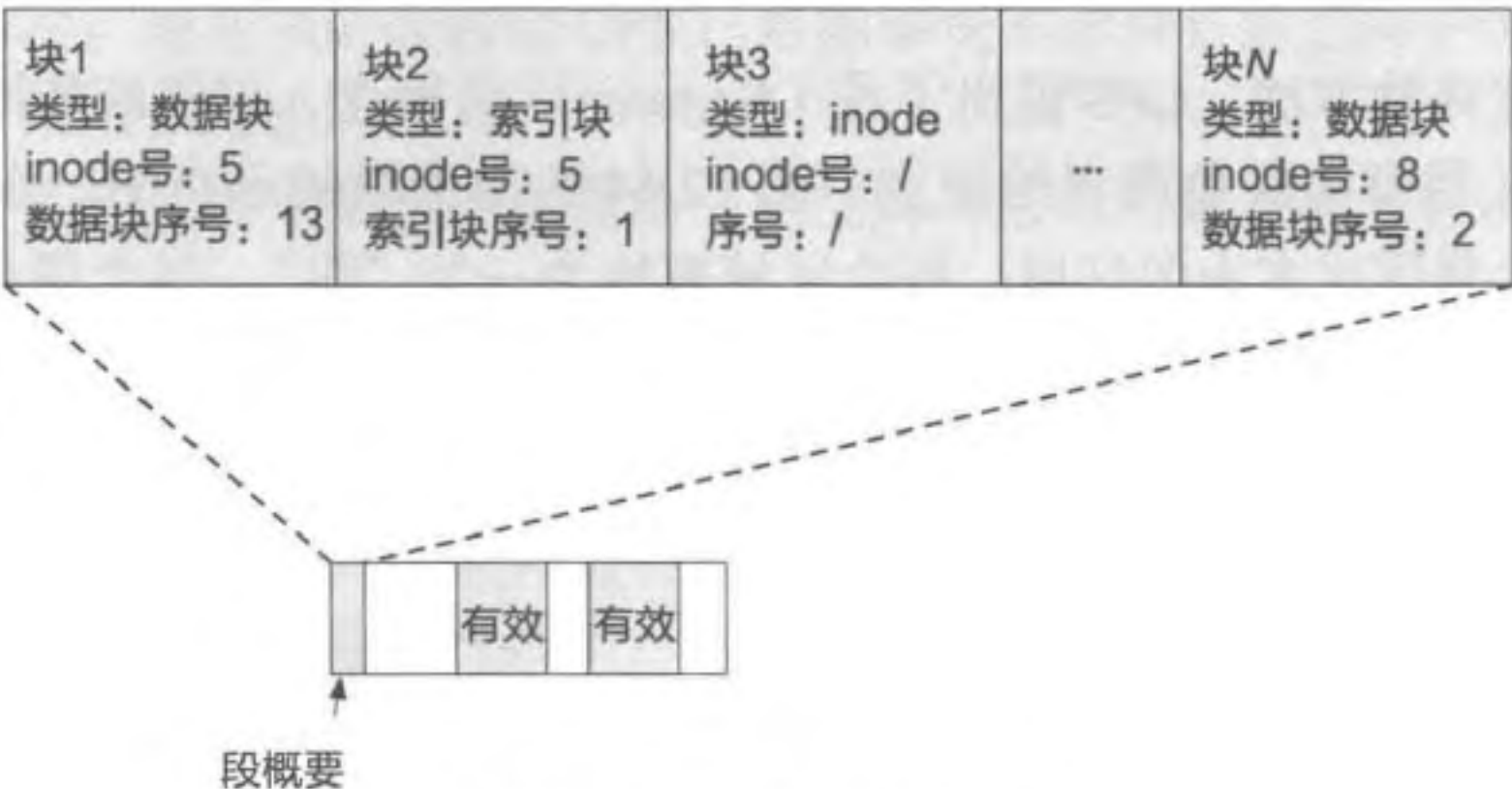


图 12.15 LFS 中的段概要

除了段概要之外，为了方便地跟踪每个段中空间的使用情况，LFS 还增加了段使用表 (Segment Usage Table)，用于记录每个段中有效数据块的数量，以及段最后被写入的时间。根据段使用表中的信息，LFS 可以快速掌握每个段的使用情况，并根据具体情况使用多种不同的策略进行段清理。

12.6.4 检查点和前滚

在前面的章节中，我们主要介绍了对文件系统的写入操作。而对于文件系统的读取请求来说，LFS 在内存结构中保存了一份文件系统结构的缓存，因此读取请求大多可以在内存中完成。由于 LFS 中的许多结构并没有固定位置，LFS 在进行挂载和崩溃恢复时，需要扫描存储空间中的所有日志，才能在内存中重构出文件系统结构的缓存。扫描整个存储空间比较耗时，造成文件系统的挂载甚至整个系统的启动时间延长。为了提升挂载和恢复的效率，LFS 使用了检查点 (Checkpoint) 和前滚 (Roll-forward) 两种技术。

检查点

在玩单机游戏时，玩家可以通过存档避免每次都重新开始游戏。类似地，日志结构文件系统可以记录一些信息，以避免每次挂载时都重新扫描和重做所有的日志。这些被记录下来的信息即为检查点。通常来说，检查点需要满足一些要求。首先，通过检查点，日志结构文件系统需要能够找到在创建检查点时文件系统中所有的有效状态（数据和元

数据);其次,这些状态需要满足文件系统的一致性要求。

LFS 创建检查点的过程分为两步:

1. 由于文件系统会在内存结构中缓存部分修改, LFS 在创建检查点时,需要先将所有的修改追加到日志中,包括文件数据块、索引块、inode 结构、inode 表和段使用表。
2. 当所有的修改均已写入日志后, LFS 在检查点区域固定的位置写入一个检查点,其内容包括 inode 表的地址、段使用表的地址、当前时间和最后一个写入的段的位置。

一旦检查点创建完毕,在进行挂载和恢复时, LFS 无须扫描整个存储空间,而只需要找到检查点,并根据检查点中记录的结构找出检查点创建时的有效数据即可。检查点避免了 LFS 对于无效数据的扫描,从而缩短了挂载和恢复的时间。

由于检查点决定了挂载时文件系统中的有效数据,检查点自身的数据完整性也十分重要。如果在创建检查点时发生崩溃,文件系统可能会使用一个不完整的检查点,从而造成文件系统格式损坏和数据丢失。为了保证检查点数据的完整性, LFS 交替使用两个不同的位置来创建检查点(如图 12.11 所示)。具体来说,若前一次创建的检查点在位置 2,则在创建新的检查点时, LFS 将数据写入检查点 1 的位置。当所有的数据都已经写入并持久化到位置 1 之后, LFS 在检查点 1 中写入当前时间。在进行挂载和恢复时, LFS 会通过比较检查点的写入时间,选择最近写入的检查点进行恢复。因此,若在检查点 1 写入的时间持久化之前发生崩溃, LFS 可以使用检查点 2 进行恢复;若在之后发生崩溃,则可以通过检查点 1 中的信息进行恢复。

一般情况下,在文件系统卸载时, LFS 会创建检查点,以保证下次挂载时无须扫描并重做日志。此外, LFS 还会每隔一段时间创建新的检查点。此处的间隔时间长短是一个权衡:如果间隔时间太长,会使得前滚的时间较长,检查点的作用被削弱;如果间隔时间太短,会频繁地创建检查点,也会对性能带来很大影响。

前滚

检查点只能将文件系统恢复到创建检查点时的状态,如果想要恢复那些在创建检查点之后写入日志的修改,还需要进行前滚操作。

前滚操作在检查点恢复之后进行,其通过扫描在检查点之后写入的日志,尽可能恢复在检查点之后进行的修改。具体来说,前滚操作会找到检查点之后修改过的段,并读取其中的段概要进行恢复。例如,如果在段概要中记录了一个新创建的、不在 inode 表中的 inode 结构,前滚操作会将新的 inode 结构添加到 inode 表中。考虑到日志的写入顺序,将 inode 结构恢复之后, inode 结构所引用的那些数据块和索引块连带被恢复。当然,此时恢复的 inode 结构可能由于目录项的丢失而处于从根开始的文件系统树之外,这就引出了另外一个问题:目录项和 inode 的一致性问题。

每个 inode 结构中记录的链接数应与保存该 inode 号的目录项的个数一致。然而当崩溃发生时,前述的 LFS 机制无法保证此一致性。例如, inode 结构已经被持久化,可是指向此 inode 的目录项并未持久化。为了保证目录操作的一致性, LFS 对每个目录操作

增加了一条特殊的记录，包括操作类型（创建、删除、重命名、增加链接）、操作所对应的目录项及其位置以及目录项中保存的 inode 结构的新链接数量，这些记录都保存在一个叫作目录修改日志的结构中。在进行文件系统操作时，LFS 会保证目录修改日志在对应的目录项或 inode 结构之前追加到日志中；在进行前滚的时候，LFS 会扫描目录修改日志，并根据其中涉及的修改是否已经全部写入日志中来对 inode 的链接数进行修复。

12.6.5 小结

本节介绍的 LFS 基于最早的 Sprite LFS 文件系统^[15]。随着存储设备的不断改进，文件系统也在不断更新换代，然而以日志形式组织文件系统的概念却一直延续至今。许多为新型硬件设计的文件系统依然使用了 LFS 的思想，在不同的硬件特性下展示出不同的优势。

闪存设备上的 LFS 便是一个很好的例子，例如被广泛用于移动设备的 F2FS^[16]。LFS 设计之初是为了减少机械磁盘设备的随机写入，从而避免频繁寻道。闪存设备在读写时，虽然不需要进行寻道操作，但其一般有较短的擦写寿命。为了防止闪存设备某些位置被写穿而导致容量变小，文件系统需要让写入操作平均分摊到设备中的各个存储单元，即进行磨损均衡（Wear Leveling）操作。而 LFS 的顺序写入，刚好可以降低闪存存储设备不同位置的写入次数差。因此，早期闪存设备上的文件系统一般都是 LFS 的形式。随着闪存设备的发展，设备的固件一般会考虑并缓解闪存的磨损问题。在这种情况下，LFS 的顺序写入思想依然符合设备的最佳访问方式，并可以充分发挥设备的并发性，提升访问闪存设备时的性能。

12.7 思考题

1. 在文件系统使用多种崩溃一致性保护方法的情况下，文件系统一致性检查工具是否还有意义？
2. 使用日志进行崩溃一致性保证时，记入日志的数据需要被写入两次：一次在日志中，另一次在其原位置上。当进行大批量的文件数据修改时，两次写入问题会严重影响文件系统的性能。请问：有何方法解决两次写入问题，使日志可以高效地保证大文件修改的一致性？
3. 写时拷贝技术和同步机制中的 RCU（Read-Copy-Update）有何相似之处，又有何区别？
4. 当使用 Soft updates 删除文件 A 时，发现存在文件 B，且文件 B 是文件 A 的硬链接。此时的删除操作与普通删除文件有何区别？如果文件 B 是文件 A 的符号链接呢？
5. 在 LFS 的基础上，如果将超级块和检查点也以日志的形式进行记录（即没有固定位置），会产生什么问题？是否可以通过设计来解决？